Artificial Intelligence and Expert System Laboratory

Course Code: MLA01

Record of Experiments

Aim | Algorithm | Python Program | Output | Result

Total experiments: 27

Index

Experiment 1: Breadth First Search (BFS) . . . 3

Experiment 2: Depth First Search (DFS) . . . 4

Experiment 3: Uniform Cost Search (UCS) . . . 5

Experiment 4: Best First Search . . . 7

Experiment 5: Greedy Best First Search . . . 8

Experiment 6: A* Search . . . 9

Experiment 7: Minimax Algorithm . . . 11

Experiment 8: Alpha-Beta Pruning . . . 12

Experiment 9: Water Jug Problem . . . 13

Experiment 10: Towers of Hanoi . . . 15

Experiment 11: Crypt Arithmetic Problem . . . 16

Experiment 12: Decision Tree Classifier . . . 17

Experiment 13: Artificial Neural Network . . . 19

Experiment 14: Diet Recommendation Expert System . . . 21

Experiment 15: Family Knowledge Representation . . . 23

Experiment 16: Birds Fly Knowledge Base . . . 25

Experiment 17: Medical Expert System . . . 26

Experiment 18: Fruit Color Classification . . . 28

Experiment 19: Planet Knowledge Representation . . . 29

Experiment 20: Person DOB Knowledge Base . . . 30

Experiment 21: Pattern Recognition . . . 31

Experiment 22: Monkey and Banana Problem . . . 33

Experiment 23: Forward Chaining . . . 35

Experiment 24: Backward Chaining . . . 36

Experiment 25: Best First Search in Prolog (Python implementation) . . . 38

Experiment 26: Sum of 1 to N using Recursion . . . 39

Experiment 27: Vowel Identification using Rule-based Logic . . . 40

Experiment 1: Breadth First Search (BFS)

Scenario: *A robot must find the shortest route to visit all rooms in a building level by level.*

AIM

To implement the Breadth First Search (BFS) algorithm to explore nodes level-wise and find the shortest path in an unweighted graph.

ALGORITHM

1. Create an empty queue and insert the start node (as a path) into it; mark the start node as visited.
2. Remove the first path from the queue and take its last node as the current node.
3. If the current node is the goal, stop and return the path (it is the shortest in an unweighted graph).
4. Otherwise, for every unvisited neighbour of the current node, mark it visited and add a new path (current path + neighbour) to the end of the queue.
5. Repeat steps 2 to 4 until the queue becomes empty or the goal is found.

PROGRAM (Python)

```
from collections import deque

# Building map: each room is connected to the rooms reachable from it
graph = {
    'Entrance': ['Hall', 'Lobby'],
    'Hall': ['Room1', 'Room2'],
    'Lobby': ['Room3'],
    'Room1': ['Room4'],
    'Room2': ['Room4'],
    'Room3': ['Room5'],
    'Room4': ['Exit'],
    'Room5': ['Exit'],
    'Exit': []
}

def bfs(graph, start, goal):
    visited = [start]
    queue = deque([[start]])
    order = []
    while queue:
        path = queue.popleft()
        node = path[-1]
        order.append(node)
        if node == goal:
            return order, path
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(path + [neighbour])
    return order, None

order, path = bfs(graph, 'Entrance', 'Exit')
print("BFS traversal order :", " -> ".join(order))
print("Shortest path       :", " -> ".join(path))
print("Number of moves     :", len(path) - 1)
```

OUTPUT

```
BFS traversal order : Entrance -> Hall -> Lobby -> Room1 -> Room2 -> Room3 -> Room4 -> Room5 -> Exit
Shortest path       : Entrance -> Hall -> Room1 -> Room4 -> Exit
Number of moves     : 4
```

RESULT

The BFS algorithm explored the building level by level and found the shortest route from the Entrance to the Exit (4 moves). Hence, BFS was implemented successfully.

Experiment 2: Depth First Search (DFS)

Scenario: *A rescue drone explores a cave by moving as deep as possible before backtracking.*

AIM

To implement the Depth First Search (DFS) algorithm to traverse the nodes using depth-first exploration.

ALGORITHM

1. Start from the source node, mark it as visited and print it.
2. Pick an unvisited neighbour of the current node and recursively apply DFS to it (go as deep as possible).
3. When a node has no unvisited neighbour, backtrack to the previous node.
4. Continue until all reachable nodes have been visited or the goal is reached.

PROGRAM (Python)

```
# Cave map: chambers and the tunnels between them
cave = {
    'Entrance': ['A', 'B'],
    'A': ['C', 'D'],
    'B': ['E'],
    'C': [],
    'D': ['F'],
    'E': ['F'],
    'F': []
}

visited = []

def dfs(node):
    visited.append(node)
    print("Visiting", node)
    for neighbour in cave[node]:
        if neighbour not in visited:
            dfs(neighbour)
    print("Backtracking from", node)

dfs('Entrance')
print("\nDFS traversal order:", " -> ".join(visited))
```

OUTPUT

```
Visiting Entrance
Visiting A
Visiting C
Backtracking from C
Visiting D
Visiting F
Backtracking from F
Backtracking from D
Backtracking from A
Visiting B
Visiting E
Backtracking from E
Backtracking from B
Backtracking from Entrance

DFS traversal order: Entrance -> A -> C -> D -> F -> B -> E
```

RESULT

The DFS algorithm visited each chamber by going as deep as possible and then backtracking. Hence, DFS was implemented successfully.

Experiment 3: Uniform Cost Search (UCS)

Scenario: *A logistics company wants to deliver goods with the minimum transportation cost.*

AIM

To implement the Uniform Cost Search (UCS) algorithm to find the least-cost path between a source and a destination.

ALGORITHM

1. Create a priority queue ordered by path cost g(n) and insert (0, source, [source]).
2. Remove the entry with the lowest cost from the priority queue.
3. If the node is already explored, skip it; otherwise mark it explored.
4. If the node is the goal, return its cost and path.
5. For every neighbour not yet explored, push (cost + edge weight, neighbour, path + neighbour) into the priority queue.
6. Repeat until the goal is found or the queue is empty.

PROGRAM (Python)

```
import heapq

# Road network: (next city, transportation cost)
graph = {
    'Warehouse': [('A', 4), ('B', 2)],
    'A': [('C', 5), ('D', 10)],
    'B': [('A', 1), ('D', 8)],
    'C': [('Customer', 3)],
    'D': [('Customer', 2)],
    'Customer': []
}

def ucs(graph, start, goal):
    pq = [(0, start, [start])]
    explored = set()
    while pq:
        cost, node, path = heapq.heappop(pq)
        if node in explored:
            continue
        explored.add(node)
        print("Expanding", node, "with cost", cost)
        if node == goal:
            return cost, path
        for nxt, weight in graph[node]:
            if nxt not in explored:
                heapq.heappush(pq, (cost + weight, nxt, path + [nxt]))
    return None, None

cost, path = ucs(graph, 'Warehouse', 'Customer')
print("\nLeast-cost path :", " -> ".join(path))
print("Total cost      :", cost)
```

OUTPUT

```
Expanding Warehouse with cost 0
Expanding B with cost 2
Expanding A with cost 3
Expanding C with cost 8
Expanding D with cost 10
Expanding Customer with cost 11

Least-cost path : Warehouse -> B -> A -> C -> Customer
Total cost      : 11
```

RESULT

UCS always expanded the cheapest node first and obtained the minimum-cost delivery route from the Warehouse to the Customer. Hence, UCS was implemented successfully.

Experiment 4: Best First Search

Scenario: *A GPS navigation system selects the next city based on the estimated distance to the destination.*

AIM

To implement the Best First Search algorithm to find a path using heuristic evaluation.

ALGORITHM

1. Place the start city in the OPEN list (a priority queue ordered by the heuristic value h(n)).
2. Remove the city with the smallest h(n) from OPEN.
3. If it is the goal city, stop and print the path.
4. Otherwise, add all its unvisited neighbours to OPEN with their h(n) values.
5. Repeat steps 2 to 4 until the goal is reached or OPEN becomes empty.

PROGRAM (Python)

```
import heapq

graph = {
    'Chennai': ['Bangalore', 'Hyderabad'],
    'Bangalore': ['Pune', 'Hyderabad'],
    'Hyderabad': ['Nagpur'],
    'Pune': ['Bhopal'],
    'Nagpur': ['Bhopal', 'Delhi'],
    'Bhopal': ['Delhi'],
    'Delhi': []
}
# Estimated straight-line distance (km) from each city to Delhi
h = {'Chennai': 1760, 'Bangalore': 1750, 'Hyderabad': 1260,
     'Pune': 1175, 'Nagpur': 1090, 'Bhopal': 600, 'Delhi': 0}

def best_first(start, goal):
    open_list = [(h[start], start, [start])]
    visited = set()
    while open_list:
        hv, city, path = heapq.heappop(open_list)
        if city in visited:
            continue
        visited.add(city)
        print("Selected: %-10s (h = %d)" % (city, hv))
        if city == goal:
            return path
        for nxt in graph[city]:
            if nxt not in visited:
                heapq.heappush(open_list, (h[nxt], nxt, path + [nxt]))
    return None

path = best_first('Chennai', 'Delhi')
print("\nPath found:", " -> ".join(path))
```

OUTPUT

```
Selected: Chennai    (h = 1760)
Selected: Hyderabad  (h = 1260)
Selected: Nagpur     (h = 1090)
Selected: Delhi      (h = 0)

Path found: Chennai -> Hyderabad -> Nagpur -> Delhi
```

RESULT

Best First Search selected the most promising city (lowest heuristic value) at every step and reached Delhi from Chennai. Hence, Best First Search was implemented successfully.

Experiment 5: Greedy Best First Search

Scenario: *A warehouse robot chooses the nearest storage rack at every step to minimize travel time.*

AIM

To implement the Greedy Best First Search algorithm to solve a pathfinding problem using a greedy heuristic.

ALGORITHM

1. Start at the robot's initial position and keep a list of racks that are still to be visited.
2. Compute the heuristic value (straight-line distance) from the current position to every unvisited rack.
3. Greedily move to the rack with the smallest distance and add that distance to the total travel.
4. Mark the rack as visited and make it the current position.
5. Repeat until all racks are visited and print the route and total distance.

PROGRAM (Python)

```
import math

racks = {'R1': (2, 3), 'R2': (5, 1), 'R3': (1, 7), 'R4': (6, 6), 'R5': (3, 5)}
position = (0, 0)          # robot starts at the origin
remaining = dict(racks)
route, total = ['Start'], 0.0

def distance(a, b):
    return math.sqrt((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2)

while remaining:
    # greedy choice: nearest rack from the current position
    nearest = min(remaining, key=lambda r: distance(position, remaining[r]))
    d = distance(position, remaining[nearest])
    print("Move to", nearest, "at", remaining[nearest], "distance = %.2f" % d)
    total += d
    position = remaining.pop(nearest)
    route.append(nearest)

print("\nRoute         :", " -> ".join(route))
print("Total distance: %.2f" % total)
```

OUTPUT

```
Move to R1 at (2, 3) distance = 3.61
Move to R5 at (3, 5) distance = 2.24
Move to R3 at (1, 7) distance = 2.83
Move to R4 at (6, 6) distance = 5.10
Move to R2 at (5, 1) distance = 5.10

Route         : Start -> R1 -> R5 -> R3 -> R4 -> R2
Total distance: 18.87
```

RESULT

The robot always moved to the nearest unvisited rack, visiting all racks using the greedy heuristic. Hence, Greedy Best First Search was implemented successfully.

Experiment 6: A* Search

Scenario: *A delivery robot must reach its destination using both actual travel cost and estimated remaining distance.*

AIM

To implement the A* Search algorithm to obtain the optimal path using the heuristic and the path cost, f(n) = g(n) + h(n).

ALGORITHM

1. Insert the start node into the OPEN list with g = 0 and f = h(start).
2. Remove the node with the lowest f(n) = g(n) + h(n) from OPEN.
3. If it is the goal node, return the path and its cost.
4. Otherwise, for each neighbour compute the new g value; if it is better than any previously found g for that neighbour, insert it into OPEN with f = g + h.
5. Repeat until the goal is reached or OPEN is empty.

PROGRAM (Python)

```
import heapq

graph = {
    'S': [('A', 1), ('B', 4)],
    'A': [('B', 2), ('C', 5)],
    'B': [('C', 2), ('D', 7)],
    'C': [('G', 3)],
    'D': [('G', 2)],
    'G': []
}
h = {'S': 7, 'A': 6, 'B': 4, 'C': 3, 'D': 2, 'G': 0}   # admissible heuristic

def a_star(start, goal):
    open_list = [(h[start], 0, start, [start])]     # (f, g, node, path)
    best_g = {start: 0}
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        if g > best_g[node]:                 # outdated (more costly) entry, skip it
            continue
        print("Expanding %s : g = %d, h = %d, f = %d" % (node, g, h[node], f))
        if node == goal:
            return g, path
        for nxt, w in graph[node]:
            new_g = g + w
            if nxt not in best_g or new_g < best_g[nxt]:
                best_g[nxt] = new_g
                heapq.heappush(open_list, (new_g + h[nxt], new_g, nxt, path + [nxt]))
    return None, None

cost, path = a_star('S', 'G')
print("\nOptimal path:", " -> ".join(path))
print("Path cost   :", cost)
```

OUTPUT

```
Expanding S : g = 0, h = 7, f = 7
Expanding A : g = 1, h = 6, f = 7
Expanding B : g = 3, h = 4, f = 7
Expanding C : g = 5, h = 3, f = 8
Expanding G : g = 8, h = 0, f = 8

Optimal path: S -> A -> B -> C -> G
Path cost   : 8
```

RESULT

A* used the actual cost g(n) together with the heuristic h(n) and found the optimal path from S to G. Hence, the A* search algorithm was implemented successfully.

Experiment 7: Minimax Algorithm

Scenario: *A chess-playing AI must determine the best move against an opponent.*

AIM

To implement the Minimax algorithm to select the optimal move through adversarial search.

ALGORITHM

1. Represent the game as a tree; leaf nodes hold the utility (score) of the final positions.
2. At a MAX node (the AI's turn) choose the child with the maximum value.
3. At a MIN node (the opponent's turn) choose the child with the minimum value.
4. Recursively evaluate the tree from the leaves up to the root.
5. The root's best child gives the optimal move for the AI.

PROGRAM (Python)

```
# Game tree of depth 3: root (MAX) -> MIN nodes -> MAX nodes -> leaf scores
tree = [
    [[3, 5], [6, 9]],      # move 0
    [[1, 2], [0, -1]]      # move 1
]

def minimax(node, is_max):
    if not isinstance(node, list):          # leaf node
        return node
    values = [minimax(child, not is_max) for child in node]
    return max(values) if is_max else min(values)

# Evaluate every move available to MAX at the root
move_values = [minimax(child, False) for child in tree]
best_move = move_values.index(max(move_values))

print("Value of each move :", move_values)
print("Optimal value      :", max(move_values))
print("Best move for MAX  : move", best_move)
```

OUTPUT

```
Value of each move : [5, 0]
Optimal value      : 5
Best move for MAX  : move 0
```

RESULT

The Minimax algorithm evaluated the complete game tree and selected the move that gives the best guaranteed score against an optimal opponent. Hence, Minimax was implemented successfully.

Experiment 8: Alpha-Beta Pruning

Scenario: *A game AI needs to make faster decisions by avoiding unnecessary game-tree exploration.*

AIM

To implement Alpha-Beta Pruning to reduce the search time while preserving the optimal decision.

ALGORITHM

1. Initialise alpha = -infinity (best value for MAX) and beta = +infinity (best value for MIN).
2. At a MAX node update alpha = max(alpha, value); at a MIN node update beta = min(beta, value).
3. If alpha >= beta at any node, prune (skip) the remaining children of that node.
4. Return the node value to its parent and continue the depth-first evaluation.
5. Count the leaves examined to show how many nodes were pruned.

PROGRAM (Python)

```
import math

tree = [
    [[3, 5], [6, 9]],
    [[1, 2], [0, -1]]
]
leaves_visited = 0

def alphabeta(node, is_max, alpha, beta):
    global leaves_visited
    if not isinstance(node, list):
        leaves_visited += 1
        return node
    if is_max:
        value = -math.inf
        for child in node:
            value = max(value, alphabeta(child, False, alpha, beta))
            alpha = max(alpha, value)
            if alpha >= beta:
                print("  Pruned remaining branches at a MAX node (alpha=%s, beta=%s)" % (alpha, beta))
                break
        return value
    else:
        value = math.inf
        for child in node:
            value = min(value, alphabeta(child, True, alpha, beta))
            beta = min(beta, value)
            if alpha >= beta:
                print("  Pruned remaining branches at a MIN node (alpha=%s, beta=%s)" % (alpha, beta))
                break
        return value

best = alphabeta(tree, True, -math.inf, math.inf)
print("Optimal value        :", best)
print("Leaves examined      :", leaves_visited, "out of 8")
print("Leaves pruned        :", 8 - leaves_visited)
```

OUTPUT

```
  Pruned remaining branches at a MAX node (alpha=6, beta=5)
  Pruned remaining branches at a MIN node (alpha=5, beta=2)
Optimal value        : 5
Leaves examined      : 5 out of 8
Leaves pruned        : 3
```

RESULT

Alpha-Beta pruning returned the same optimal value as Minimax while examining fewer leaf nodes. Hence, Alpha-Beta Pruning was implemented successfully.

Experiment 9: Water Jug Problem

Scenario: *A person has two containers of different capacities and must measure an exact quantity of water.*

AIM

To solve the Water Jug Problem (4-litre and 3-litre jugs, goal = exactly 2 litres in the 4-litre jug) and generate the sequence of operations to reach the goal state.

ALGORITHM

1. Represent a state as (x, y): the water in the 4-litre jug and in the 3-litre jug. The start state is (0, 0).
2. Apply the operations: fill a jug, empty a jug, pour from one jug to the other until the receiver is full or the giver is empty.
3. Use BFS over the states, storing the parent state and operation used to reach every new state.
4. Stop when the state (2, y) is reached.
5. Trace back through the parents to print the sequence of operations.

PROGRAM (Python)

```
from collections import deque

CAP_X, CAP_Y, GOAL = 4, 3, 2

def next_states(x, y):
    pour_xy = min(x, CAP_Y - y)
    pour_yx = min(y, CAP_X - x)
    return [
        ((CAP_X, y), "Fill the 4L jug"),
        ((x, CAP_Y), "Fill the 3L jug"),
        ((0, y), "Empty the 4L jug"),
        ((x, 0), "Empty the 3L jug"),
        ((x - pour_xy, y + pour_xy), "Pour 4L jug -> 3L jug"),
        ((x + pour_yx, y - pour_yx), "Pour 3L jug -> 4L jug"),
    ]

def solve():
    start = (0, 0)
    parent = {start: (None, "Start")}
    queue = deque([start])
    while queue:
        state = queue.popleft()
        if state[0] == GOAL:
            steps = []
            while state is not None:
                prev, op = parent[state]
                steps.append((op, state))
                state = prev
            return steps[::-1]
        for new_state, op in next_states(*state):
            if new_state not in parent:
                parent[new_state] = (state, op)
                queue.append(new_state)

for i, (op, state) in enumerate(solve()):
    print("Step %d: %-24s -> state (4L, 3L) = %s" % (i, op, state))
```

OUTPUT

```
Step 0: Start                    -> state (4L, 3L) = (0, 0)
Step 1: Fill the 4L jug          -> state (4L, 3L) = (4, 0)
Step 2: Pour 4L jug -> 3L jug    -> state (4L, 3L) = (1, 3)
Step 3: Empty the 3L jug         -> state (4L, 3L) = (1, 0)
Step 4: Pour 4L jug -> 3L jug    -> state (4L, 3L) = (0, 1)
Step 5: Fill the 4L jug          -> state (4L, 3L) = (4, 1)
Step 6: Pour 4L jug -> 3L jug    -> state (4L, 3L) = (2, 3)
```

RESULT

The goal of exactly 2 litres in the 4-litre jug was reached using the shortest sequence of operations shown above. Hence, the Water Jug Problem was solved successfully.

Experiment 10: Towers of Hanoi

Scenario: *A robotic arm transfers stacked containers while following movement constraints.*

AIM

To solve the Towers of Hanoi problem and generate the optimal sequence of moves.

ALGORITHM

1. There are three pegs (Source, Auxiliary, Destination) and n disks stacked in decreasing size on the Source peg.
2. Only one disk may be moved at a time and a larger disk may never be placed on a smaller disk.
3. If n = 1, move the disk directly from Source to Destination.
4. Otherwise, move the top n-1 disks from Source to Auxiliary, move the largest disk from Source to Destination, then move the n-1 disks from Auxiliary to Destination.
5. The minimum number of moves required is 2^n - 1.

PROGRAM (Python)

```
moves = 0

def hanoi(n, source, auxiliary, destination):
    global moves
    if n == 1:
        moves += 1
        print("Move %d: disk 1 from %s to %s" % (moves, source, destination))
        return
    hanoi(n - 1, source, destination, auxiliary)
    moves += 1
    print("Move %d: disk %d from %s to %s" % (moves, n, source, destination))
    hanoi(n - 1, auxiliary, source, destination)

n = 3
hanoi(n, 'A', 'B', 'C')
print("\nTotal moves for %d disks = %d (2^%d - 1 = %d)" % (n, moves, n, 2 ** n - 1))
```

OUTPUT

```
Move 1: disk 1 from A to C
Move 2: disk 2 from A to B
Move 3: disk 1 from C to B
Move 4: disk 3 from A to C
Move 5: disk 1 from B to A
Move 6: disk 2 from B to C
Move 7: disk 1 from A to C

Total moves for 3 disks = 7 (2^3 - 1 = 7)
```

RESULT

The 3 disks were transferred from peg A to peg C in the minimum number of moves (7). Hence, the Towers of Hanoi problem was solved successfully.

Experiment 11: Crypt Arithmetic Problem

Scenario: *A banking application verifies encrypted arithmetic puzzles before processing transactions.*

AIM

To solve the crypt-arithmetic puzzle SEND + MORE = MONEY by assigning valid digits that satisfy the arithmetic constraints.

ALGORITHM

1. Collect the distinct letters of the puzzle (S, E, N, D, M, O, R, Y); there are 8 letters and 10 digits.
2. Generate every possible assignment of distinct digits to the letters.
3. Reject an assignment if the first letter of any word (S or M) is 0.
4. Convert the words to numbers and test whether SEND + MORE = MONEY.
5. Print the first assignment that satisfies the equation.

PROGRAM (Python)

```
from itertools import permutations

letters = "SENDMORY"

def value(word, assign):
    number = 0
    for ch in word:
        number = number * 10 + assign[ch]
    return number

def solve():
    for digits in permutations(range(10), len(letters)):
        assign = dict(zip(letters, digits))
        if assign['S'] == 0 or assign['M'] == 0:
            continue
        if value("SEND", assign) + value("MORE", assign) == value("MONEY", assign):
            return assign
    return None

solution = solve()
print("Letter-digit assignment:", solution)
print("  %d" % value("SEND", solution))
print("+ %d" % value("MORE", solution))
print("-------")
print("%d" % value("MONEY", solution))
```

OUTPUT

```
Letter-digit assignment: {'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
  9567
+ 1085
-------
10652
```

RESULT

A valid assignment of digits to letters satisfying SEND + MORE = MONEY was found. Hence, the crypt-arithmetic problem was solved successfully.

Experiment 12: Decision Tree Classifier

Scenario: *A bank wants to approve or reject loan applications based on customer information.*

AIM

To build a Decision Tree Classifier that classifies loan applications as approved or rejected using decision tree learning.

ALGORITHM

1. Collect the dataset with attributes Age, Income, Credit Score and Employed, and the class label Approved (1) or Rejected (0).
2. Split the dataset into a training set (75%) and a test set (25%).
3. Build the tree: select the attribute with the highest information gain (entropy criterion) at each node and split the data on it.
4. Repeat recursively until every leaf node is pure.
5. Test the tree on the test set, compute the accuracy, and predict the decision for new applicants.

PROGRAM (Python)

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

features = ['Age', 'Income(k)', 'CreditScore', 'Employed']
# Age, Income (in thousands), Credit score, Employed (1/0), Approved (1/0)
data = [
    [25, 30, 600, 1, 0], [35, 60, 720, 1, 1], [45, 80, 750, 1, 1],
    [28, 25, 580, 0, 0], [52, 90, 780, 1, 1], [23, 20, 550, 0, 0],
    [40, 55, 690, 1, 1], [33, 40, 640, 1, 0], [31, 45, 700, 1, 1],
    [29, 35, 610, 1, 0], [48, 70, 710, 0, 0], [55, 95, 800, 1, 1],
    [38, 50, 660, 1, 0], [42, 65, 730, 1, 1], [26, 28, 590, 0, 0],
    [36, 58, 705, 1, 1],
]
X = [row[:4] for row in data]
y = [row[4] for row in data]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)

model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train, y_train)

print("Learned decision tree:")
print(export_text(model, feature_names=features))
print("Test accuracy:", accuracy_score(y_test, model.predict(X_test)))

new_applicants = [[30, 50, 710, 1], [27, 22, 570, 0]]
for person, label in zip(new_applicants, model.predict(new_applicants)):
    print("Applicant", person, "->", "APPROVED" if label == 1 else "REJECTED")
```

OUTPUT

```
Learned decision tree:
|--- CreditScore <= 675.00
|   |--- class: 0
|--- CreditScore >  675.00
|   |--- Employed <= 0.50
|   |   |--- class: 0
|   |--- Employed >  0.50
|   |   |--- class: 1

Test accuracy: 1.0
Applicant [30, 50, 710, 1] -> APPROVED
Applicant [27, 22, 570, 0] -> REJECTED
```

RESULT

The decision tree was trained on the loan data, tested on unseen records (test accuracy 100% on the small test set) and used to approve or reject new applicants. Hence, the Decision Tree Classifier was implemented successfully.

Experiment 13: Artificial Neural Network

Scenario: *A hospital predicts whether a patient has a disease using historical medical records.*

AIM

To implement an Artificial Neural Network (multi-layer perceptron) and train and test it on medical data.

ALGORITHM

1. Load the medical dataset (Breast Cancer Wisconsin dataset: 569 patients, 30 features, class malignant/benign).
2. Split the data into training (80%) and testing (20%) sets and scale the features.
3. Create a neural network with an input layer, two hidden layers (16 and 8 neurons, ReLU) and an output layer.
4. Train the network using forward propagation, error calculation and back-propagation of the error to update the weights.
5. Test the trained model on the test set and report the accuracy and confusion matrix.

PROGRAM (Python)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42, stratify=data.target)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

ann = MLPClassifier(hidden_layer_sizes=(16, 8), activation='relu',
                    max_iter=500, random_state=42)
ann.fit(X_train, y_train)

pred = ann.predict(X_test)
print("Training samples:", len(X_train), " Testing samples:", len(X_test))
print("Training iterations:", ann.n_iter_)
print("Test accuracy: %.4f" % accuracy_score(y_test, pred))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, target_names=data.target_names))
```

OUTPUT

```
Training samples: 455  Testing samples: 114
Training iterations: 345
Test accuracy: 0.9561
Confusion matrix:
 [[41  1]
 [ 4 68]]
              precision    recall  f1-score   support

   malignant       0.91      0.98      0.94        42
      benign       0.99      0.94      0.96        72

    accuracy                           0.96       114
   macro avg       0.95      0.96      0.95       114
weighted avg       0.96      0.96      0.96       114
```

RESULT

The neural network was trained on historical patient records and predicted the disease class of unseen patients with a test accuracy of 95.61%. Hence, the Artificial Neural Network was implemented successfully.

Experiment 14: Diet Recommendation Expert System

Scenario: *A nutrition application recommends diet plans based on age, BMI, and health conditions.*

AIM

To develop a Diet Recommendation Expert System that recommends personalised diet plans using rules.

ALGORITHM

1. Take the age, weight (kg), height (m) and health condition of the person as input.
2. Calculate BMI = weight / (height x height) and classify it as Underweight, Normal, Overweight or Obese.
3. Apply the IF-THEN rules of the knowledge base: health-condition rules (diabetes, hypertension) have the highest priority, followed by BMI rules and then age rules.
4. Collect the advice given by every rule that fires.
5. Display the BMI category and the recommended diet plan.

PROGRAM (Python)

```
def bmi_category(bmi):
    if bmi < 18.5:
        return "Underweight"
    elif bmi < 25:
        return "Normal"
    elif bmi < 30:
        return "Overweight"
    return "Obese"

def recommend(name, age, weight, height, condition):
    bmi = weight / (height * height)
    category = bmi_category(bmi)
    advice = []
    # Rules on health condition
    if condition == "diabetes":
        advice.append("Low sugar, high fibre diet; whole grains, vegetables, no sweets")
    elif condition == "hypertension":
        advice.append("Low salt (DASH) diet; fruits, vegetables, avoid pickles and processed food")
    # Rules on BMI
    if category == "Underweight":
        advice.append("High-calorie, protein-rich diet: milk, nuts, eggs, pulses")
    elif category == "Overweight":
        advice.append("Calorie-controlled diet: more vegetables, less fried food")
    elif category == "Obese":
        advice.append("Strict low-calorie diet with daily exercise; consult a dietician")
    else:
        advice.append("Balanced diet: cereals, pulses, vegetables, fruits, milk")
    # Rules on age
    if age < 18:
        advice.append("Add calcium and iron rich foods for growth")
    elif age > 60:
        advice.append("Easily digestible food with extra calcium and vitamin D")
    print("Name: %s | Age: %d | BMI: %.1f (%s) | Condition: %s" %
          (name, age, bmi, category, condition))
    for rule in advice:
        print("  -", rule)
    print()

recommend("Arun", 25, 50, 1.75, "none")
recommend("Meena", 45, 82, 1.60, "diabetes")
recommend("Ravi", 65, 70, 1.70, "hypertension")
recommend("Kiran", 15, 60, 1.55, "none")
```

OUTPUT

```
Name: Arun | Age: 25 | BMI: 16.3 (Underweight) | Condition: none
  - High-calorie, protein-rich diet: milk, nuts, eggs, pulses

Name: Meena | Age: 45 | BMI: 32.0 (Obese) | Condition: diabetes
  - Low sugar, high fibre diet; whole grains, vegetables, no sweets
  - Strict low-calorie diet with daily exercise; consult a dietician

Name: Ravi | Age: 65 | BMI: 24.2 (Normal) | Condition: hypertension
  - Low salt (DASH) diet; fruits, vegetables, avoid pickles and processed food
  - Balanced diet: cereals, pulses, vegetables, fruits, milk
  - Easily digestible food with extra calcium and vitamin D

Name: Kiran | Age: 15 | BMI: 25.0 (Normal) | Condition: none
  - Balanced diet: cereals, pulses, vegetables, fruits, milk
  - Add calcium and iron rich foods for growth
```

RESULT

The expert system applied its rules to each person's age, BMI and health condition and recommended a suitable diet plan. Hence, the Diet Recommendation Expert System was developed successfully.

Experiment 15: Family Knowledge Representation

Scenario: *A genealogy application determines relationships among family members.*

AIM

To implement a family knowledge base and infer family relationships using logical rules.

ALGORITHM

1. Store the basic facts: male(X), female(X) and parent(X, Y), meaning X is a parent of Y.
2. Define the rules: father, mother, grandparent, sibling, brother, sister, uncle, aunt and cousin in terms of the basic facts.
3. To answer a query, evaluate the corresponding rule over all the facts and collect every person that satisfies it.
4. Display the inferred relationships.

PROGRAM (Python)

```
male = {"Rajan", "Arun", "Kiran", "Suresh", "Anand"}
female = {"Latha", "Meena", "Priya", "Divya"}
# parent(X, Y) : X is a parent of Y
parent = {
    ("Rajan", "Arun"), ("Latha", "Arun"), ("Rajan", "Meena"), ("Latha", "Meena"),
    ("Arun", "Kiran"), ("Priya", "Kiran"), ("Arun", "Divya"), ("Priya", "Divya"),
    ("Meena", "Anand"), ("Suresh", "Anand"),
}
people = male | female

def father(x, y):      return (x, y) in parent and x in male
def mother(x, y):      return (x, y) in parent and x in female
def grandparent(x, z): return any((x, y) in parent and (y, z) in parent for y in people)
def sibling(x, y):
    return x != y and any((p, x) in parent and (p, y) in parent for p in people)
def brother(x, y):     return sibling(x, y) and x in male
def sister(x, y):      return sibling(x, y) and x in female
def uncle(x, y):       return any(brother(x, p) and (p, y) in parent for p in people)
def aunt(x, y):        return any(sister(x, p) and (p, y) in parent for p in people)
def cousin(x, y):
    return any((p, x) in parent and (q, y) in parent and sibling(p, q)
               for p in people for q in people)

def query(relation, name, who_is_second=True):
    result = sorted(p for p in people if relation(p, name))
    return result

print("Father of Arun        :", query(father, "Arun"))
print("Mother of Kiran       :", query(mother, "Kiran"))
print("Grandparents of Kiran :", query(grandparent, "Kiran"))
print("Siblings of Kiran     :", query(sibling, "Kiran"))
print("Sister of Kiran       :", query(sister, "Kiran"))
print("Uncle of Anand        :", query(uncle, "Anand"))
print("Aunt of Kiran         :", query(aunt, "Kiran"))
print("Cousins of Anand      :", query(cousin, "Anand"))
```

OUTPUT

```
Father of Arun        : ['Rajan']
Mother of Kiran       : ['Priya']
Grandparents of Kiran : ['Latha', 'Rajan']
Siblings of Kiran     : ['Divya']
Sister of Kiran       : ['Divya']
Uncle of Anand        : ['Arun']
Aunt of Kiran         : ['Meena']
Cousins of Anand      : ['Divya', 'Kiran']
```

RESULT

Family relationships such as father, mother, grandparent, sibling, uncle, aunt and cousin were inferred from the basic facts using logical rules. Hence, the family knowledge representation was implemented successfully.

Experiment 16: Birds Fly Knowledge Base

Scenario: *A wildlife information system determines whether an animal can fly based on its characteristics.*

AIM

To implement the Birds Fly knowledge base and perform inheritance-based reasoning (with exceptions).

ALGORITHM

1. Store the class hierarchy as is-a links (sparrow is a bird, bird is an animal, etc.).
2. Store properties (can_fly) at the most general class, and exceptions at specific classes (penguin, ostrich).
3. To find a property of an object, look at the object itself; if it is not defined, move up to its parent class and repeat (inheritance).
4. The first (most specific) definition found is the answer; if none is found, the answer is unknown.
5. Query the knowledge base for different animals and print the reasoning chain.

PROGRAM (Python)

```
is_a = {
    "tweety": "sparrow", "sparrow": "bird", "eagle": "bird",
    "penguin": "bird", "ostrich": "bird", "pingu": "penguin",
    "bat": "mammal", "dog": "mammal", "bird": "animal", "mammal": "animal",
}
can_fly = {
    "bird": True,       # default rule: birds fly
    "penguin": False,   # exception
    "ostrich": False,   # exception
    "bat": True,        # a mammal that flies
    "mammal": False,    # default rule: mammals do not fly
}

def flies(thing):
    chain = [thing]
    current = thing
    while current is not None:
        if current in can_fly:
            return can_fly[current], current, chain
        current = is_a.get(current)
        if current:
            chain.append(current)
    return None, None, chain

for animal in ["tweety", "eagle", "pingu", "ostrich", "bat", "dog"]:
    answer, source, chain = flies(animal)
    text = "CAN fly" if answer else "CANNOT fly"
    print("%-7s %-10s (rule from '%s'; chain: %s)" %
          (animal, text, source, " -> ".join(chain)))
```

OUTPUT

```
tweety  CAN fly    (rule from 'bird'; chain: tweety -> sparrow -> bird)
eagle   CAN fly    (rule from 'bird'; chain: eagle -> bird)
pingu   CANNOT fly (rule from 'penguin'; chain: pingu -> penguin)
ostrich CANNOT fly (rule from 'ostrich'; chain: ostrich)
bat     CAN fly    (rule from 'bat'; chain: bat)
dog     CANNOT fly (rule from 'mammal'; chain: dog -> mammal)
```

RESULT

By inheritance, birds were found to fly by default while the exceptions (penguin, ostrich) and non-flying mammals were identified correctly. Hence, the Birds Fly knowledge base was implemented successfully.

Experiment 17: Medical Expert System

Scenario: *A healthcare expert system assists doctors by identifying diseases from patient symptoms.*

AIM

To develop a Medical Expert System that infers diseases using rule-based reasoning.

ALGORITHM

1. Build a knowledge base of rules: IF (set of symptoms) THEN disease.
2. Accept the symptoms reported by the patient as the working memory (facts).
3. Match the patient's symptoms against the symptoms of every rule and compute the matching percentage.
4. Select the disease with the highest matching percentage (at least 60% is required for a diagnosis).
5. Display the diagnosis with its confidence and a recommendation.

PROGRAM (Python)

```
rules = {
    "Common Cold":  {"sneezing", "runny_nose", "sore_throat", "cough"},
    "Influenza":    {"fever", "body_ache", "cough", "fatigue", "headache"},
    "Malaria":      {"fever", "chills", "sweating", "headache", "vomiting"},
    "Dengue":       {"high_fever", "rash", "joint_pain", "headache", "pain_behind_eyes"},
    "Migraine":     {"headache", "nausea", "light_sensitivity"},
}
advice = {
    "Common Cold": "Rest, warm fluids and steam inhalation.",
    "Influenza": "Rest, fluids and consult a doctor if fever persists.",
    "Malaria": "Get a blood smear test immediately and consult a doctor.",
    "Dengue": "Get a platelet count / NS1 test and consult a doctor.",
    "Migraine": "Rest in a dark room and consult a neurologist if frequent.",
}

def diagnose(symptoms):
    symptoms = set(symptoms)
    scores = []
    for disease, required in rules.items():
        match = len(symptoms & required) / len(required) * 100
        scores.append((match, disease))
    scores.sort(reverse=True)
    best_score, best_disease = scores[0]
    print("Symptoms:", sorted(symptoms))
    for score, disease in scores:
        print("   %-12s match = %5.1f%%" % (disease, score))
    if best_score >= 60:
        print("=> Diagnosis: %s (confidence %.0f%%)" % (best_disease, best_score))
        print("   Advice   :", advice[best_disease])
    else:
        print("=> No disease matched with enough confidence. Consult a doctor.")
    print()

diagnose(["fever", "chills", "sweating", "headache"])
diagnose(["sneezing", "runny_nose", "cough"])
diagnose(["headache", "nausea", "light_sensitivity"])
```

OUTPUT

```
Symptoms: ['chills', 'fever', 'headache', 'sweating']
   Malaria      match =  80.0%
   Influenza    match =  40.0%
   Migraine     match =  33.3%
   Dengue       match =  20.0%
   Common Cold  match =   0.0%
=> Diagnosis: Malaria (confidence 80%)
   Advice   : Get a blood smear test immediately and consult a doctor.

Symptoms: ['cough', 'runny_nose', 'sneezing']
   Common Cold  match =  75.0%
   Influenza    match =  20.0%
   Migraine     match =   0.0%
   Malaria      match =   0.0%
   Dengue       match =   0.0%
=> Diagnosis: Common Cold (confidence 75%)
   Advice   : Rest, warm fluids and steam inhalation.

Symptoms: ['headache', 'light_sensitivity', 'nausea']
   Migraine     match = 100.0%
   Malaria      match =  20.0%
   Influenza    match =  20.0%
   Dengue       match =  20.0%
   Common Cold  match =   0.0%
=> Diagnosis: Migraine (confidence 100%)
   Advice   : Rest in a dark room and consult a neurologist if frequent.
```

RESULT

The expert system matched the reported symptoms against its rules and inferred the most probable disease with a recommendation. Hence, the Medical Expert System was developed successfully.

Experiment 18: Fruit Color Classification

Scenario: *A fruit-sorting machine identifies fruits based on their colors.*

AIM

To implement fruit classification based on colour (and size where needed) using production rules.

ALGORITHM

1. Create a rule base of production rules of the form IF (colour = c AND other conditions) THEN fruit.
2. Read the colour (and size) detected by the sorting machine as the fact.
3. Scan the rules in order; fire the first rule whose conditions are all satisfied.
4. If no rule matches, report the fruit as unknown.
5. Print the classification for each input.

PROGRAM (Python)

```
# Production rules: (conditions, conclusion)
rules = [
    ({"color": "red", "size": "small"}, "Cherry"),
    ({"color": "red", "size": "medium"}, "Apple"),
    ({"color": "red", "size": "large"}, "Watermelon (inside)"),
    ({"color": "yellow", "size": "medium"}, "Banana"),
    ({"color": "yellow", "size": "large"}, "Pineapple"),
    ({"color": "orange"}, "Orange"),
    ({"color": "green", "size": "small"}, "Grapes"),
    ({"color": "green", "size": "medium"}, "Guava"),
    ({"color": "purple"}, "Plum / Black Grapes"),
]

def classify(fact):
    for conditions, fruit in rules:
        if all(fact.get(key) == value for key, value in conditions.items()):
            return fruit
    return "Unknown fruit"

inputs = [
    {"color": "red", "size": "medium"},
    {"color": "yellow", "size": "medium"},
    {"color": "orange", "size": "medium"},
    {"color": "green", "size": "small"},
    {"color": "purple", "size": "small"},
    {"color": "blue", "size": "small"},
]
for fact in inputs:
    print("Input", fact, "=>", classify(fact))
```

OUTPUT

```
Input {'color': 'red', 'size': 'medium'} => Apple
Input {'color': 'yellow', 'size': 'medium'} => Banana
Input {'color': 'orange', 'size': 'medium'} => Orange
Input {'color': 'green', 'size': 'small'} => Grapes
Input {'color': 'purple', 'size': 'small'} => Plum / Black Grapes
Input {'color': 'blue', 'size': 'small'} => Unknown fruit
```

RESULT

The fruits were classified using production rules based on their colour and size, and unknown inputs were reported correctly. Hence, the fruit colour classification was implemented successfully.

Experiment 19: Planet Knowledge Representation

Scenario: *An astronomy application answers questions about planets in the solar system.*

AIM

To implement planet knowledge representation and retrieve planetary facts using logical inference.

ALGORITHM

1. Store facts about each planet as a frame: position from the Sun, type, diameter, orbital period and rings.
2. Define rules as functions over the facts (for example: inner planet, largest planet, planets of a given type).
3. Convert each user question into a rule/query and evaluate it on the fact base.
4. Print the answers obtained by inference.

PROGRAM (Python)

```
planets = {
    "Mercury": {"order": 1, "type": "terrestrial", "diameter_km": 4879,   "period_days": 88,    "rings": False},
    "Venus":   {"order": 2, "type": "terrestrial", "diameter_km": 12104,  "period_days": 225,   "rings": False},
    "Earth":   {"order": 3, "type": "terrestrial", "diameter_km": 12742,  "period_days": 365,   "rings": False},
    "Mars":    {"order": 4, "type": "terrestrial", "diameter_km": 6779,   "period_days": 687,   "rings": False},
    "Jupiter": {"order": 5, "type": "gas giant",   "diameter_km": 139820, "period_days": 4333,  "rings": True},
    "Saturn":  {"order": 6, "type": "gas giant",   "diameter_km": 116460, "period_days": 10759, "rings": True},
    "Uranus":  {"order": 7, "type": "ice giant",   "diameter_km": 50724,  "period_days": 30687, "rings": True},
    "Neptune": {"order": 8, "type": "ice giant",   "diameter_km": 49244,  "period_days": 60190, "rings": True},
}

def planet_at(order):
    return [p for p, f in planets.items() if f["order"] == order][0]

def is_inner_planet(p):
    return planets[p]["order"] <= 4           # rule: inner planets are the first four

def largest():
    return max(planets, key=lambda p: planets[p]["diameter_km"])

def with_rings():
    return [p for p, f in planets.items() if f["rings"]]

def of_type(t):
    return [p for p, f in planets.items() if f["type"] == t]

print("Which planet is 3rd from the Sun?       :", planet_at(3))
print("Is Mars an inner planet?                :", is_inner_planet("Mars"))
print("Is Saturn an inner planet?              :", is_inner_planet("Saturn"))
print("Largest planet                          :", largest())
print("Planets with rings                      :", with_rings())
print("Terrestrial planets                     :", of_type("terrestrial"))
print("Year length of Mars (Earth days)        :", planets["Mars"]["period_days"])
print("Is Jupiter bigger than Earth?           :",
      planets["Jupiter"]["diameter_km"] > planets["Earth"]["diameter_km"])
```

OUTPUT

```
Which planet is 3rd from the Sun?       : Earth
Is Mars an inner planet?                : True
Is Saturn an inner planet?              : False
Largest planet                          : Jupiter
Planets with rings                      : ['Jupiter', 'Saturn', 'Uranus', 'Neptune']
Terrestrial planets                     : ['Mercury', 'Venus', 'Earth', 'Mars']
Year length of Mars (Earth days)        : 687
Is Jupiter bigger than Earth?           : True
```

RESULT

The planetary facts were stored as a knowledge base and the queries were answered by logical inference. Hence, the planet knowledge representation was implemented successfully.

Experiment 20: Person DOB Knowledge Base

Scenario: *A citizen information portal retrieves personal details using date of birth records.*

AIM

To implement a Person-DOB knowledge base and query the stored facts using logical reasoning.

ALGORITHM

1. Store the facts dob(person, date) for every citizen.
2. Define rules: date of birth of a person, persons born in a given year or month, age on a given date, and the elder of two persons.
3. Convert each query into the corresponding rule and evaluate it over the stored facts.
4. Print the results of the queries.

PROGRAM (Python)

```
from datetime import date

dob = {
    "Arun":  date(2001, 5, 14),
    "Meena": date(1998, 11, 2),
    "Kiran": date(2001, 8, 23),
    "Divya": date(2005, 5, 30),
    "Ravi":  date(1975, 1, 9),
}
TODAY = date(2026, 10, 14)       # reference date used to compute the age

def age(person):
    d = dob[person]
    return TODAY.year - d.year - ((TODAY.month, TODAY.day) < (d.month, d.day))

def born_in_year(year):
    return sorted(p for p, d in dob.items() if d.year == year)

def born_in_month(month):
    return sorted(p for p, d in dob.items() if d.month == month)

def elder(a, b):
    return a if dob[a] < dob[b] else b

print("DOB of Meena            :", dob["Meena"].strftime("%d-%m-%Y"))
print("Age of Arun             :", age("Arun"), "years")
print("Persons born in 2001    :", born_in_year(2001))
print("Persons born in May     :", born_in_month(5))
print("Elder of Arun and Kiran :", elder("Arun", "Kiran"))
print("Oldest person           :", min(dob, key=dob.get))
print("Is Divya a minor (<18)? :", age("Divya") < 18)
```

OUTPUT

```
DOB of Meena            : 02-11-1998
Age of Arun             : 25 years
Persons born in 2001    : ['Arun', 'Kiran']
Persons born in May     : ['Arun', 'Divya']
Elder of Arun and Kiran : Arun
Oldest person           : Ravi
Is Divya a minor (<18)? : False
```

RESULT

The date-of-birth facts were stored and queried using logical rules to obtain age, year-wise and month-wise details. Hence, the Person DOB knowledge base was implemented successfully.

Experiment 21: Pattern Recognition

Scenario: *An image-processing system identifies recurring visual patterns automatically.*

AIM

To implement pattern recognition and detect patterns using AI techniques (template matching to find recurring patterns in an image, and a k-Nearest Neighbour classifier to recognise handwritten digits).

ALGORITHM

1. Part A - Template matching: represent the image as a binary matrix and the pattern as a small matrix.
2. Slide the template over every position of the image, compare the pixels, and record the positions where all pixels match.
3. Count and display the occurrences of the recurring pattern.
4. Part B - Digit recognition: load the 8x8 handwritten digit images and split them into training and testing sets.
5. Train the k-Nearest Neighbour classifier (k = 3), which labels an image by the majority class of its 3 closest training images; test it and report the accuracy.

PROGRAM (Python)

```
import numpy as np

# ---------- Part A : find a recurring pattern in an image ----------
image = np.zeros((8, 12), dtype=int)
plus = np.array([[0, 1, 0],
                 [1, 1, 1],
                 [0, 1, 0]])
for r, c in [(0, 0), (1, 8), (4, 4)]:           # draw the pattern three times
    image[r:r + 3, c:c + 3] = plus

print("Image:")
for row in image:
    print(" ".join("#" if v else "." for v in row))

found = []
h, w = plus.shape
for r in range(image.shape[0] - h + 1):
    for c in range(image.shape[1] - w + 1):
        if np.array_equal(image[r:r + h, c:c + w], plus):
            found.append((r, c))
print("\nPattern occurs %d times at (row, col): %s" % (len(found), found))

# ---------- Part B : recognise handwritten digits with k-NN ----------
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(
    digits.data, digits.target, test_size=0.25, random_state=42)
knn = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)
pred = knn.predict(X_test)
print("\nDigit recognition accuracy: %.4f" % accuracy_score(y_test, pred))
print("First 10 actual   :", y_test[:10].tolist())
print("First 10 predicted:", pred[:10].tolist())
```

OUTPUT

```
Image:
. # . . . . . . . . . .
# # # . . . . . . # . .
. # . . . . . . # # # .
. . . . . . . . . # . .
. . . . . # . . . . . .
. . . . # # # . . . . .
. . . . . # . . . . . .
. . . . . . . . . . . .

Pattern occurs 3 times at (row, col): [(0, 0), (1, 8), (4, 4)]

Digit recognition accuracy: 0.9867
First 10 actual   : [6, 9, 3, 7, 2, 1, 5, 2, 5, 2]
First 10 predicted: [6, 9, 3, 7, 2, 1, 5, 2, 5, 2]
```

RESULT

The recurring pattern was detected at all of its positions in the image and the handwritten digits were recognised by the k-NN classifier with an accuracy of 98.67%. Hence, pattern recognition was implemented successfully.

Experiment 22: Monkey and Banana Problem

Scenario: *A robot must retrieve food placed on a high shelf by planning a sequence of actions.*

AIM

To solve the Monkey and Banana problem and generate an optimal action plan.

ALGORITHM

1. Represent a state as (monkey position, box position, monkey on box?, has banana?). The banana hangs at position C (centre); the monkey starts at A (door) and the box is at B (window).
2. Define the actions: walk to a place, push the box to a place, climb on the box, and grasp the banana (only possible when the monkey is on the box standing at C).
3. Apply BFS from the start state so that the first plan found has the minimum number of actions.
4. Stop when the monkey has the banana and print the plan.

PROGRAM (Python)

```
from collections import deque

PLACES = ['A', 'B', 'C']        # A = door, B = window, C = centre (banana is here)
BANANA = 'C'
NAMES = {'A': 'door', 'B': 'window', 'C': 'centre'}

def actions(state):
    monkey, box, on_box, has = state
    result = []
    if not on_box:
        for p in PLACES:
            if p != monkey:
                result.append(("Walk to the " + NAMES[p], (p, box, False, has)))
        if monkey == box:
            for p in PLACES:
                if p != box:
                    result.append(("Push the box to the " + NAMES[p], (p, p, False, has)))
            result.append(("Climb on the box", (monkey, box, True, has)))
    elif monkey == BANANA:
        result.append(("Grasp the banana", (monkey, box, True, True)))
    return result

def plan(start):
    queue = deque([start])
    parent = {start: (None, "Start")}
    while queue:
        state = queue.popleft()
        if state[3]:
            steps = []
            while state is not None:
                prev, act = parent[state]
                steps.append(act)
                state = prev
            return steps[::-1][1:]
        for act, new_state in actions(state):
            if new_state not in parent:
                parent[new_state] = (state, act)
                queue.append(new_state)

start = ('A', 'B', False, False)
solution = plan(start)
print("Initial state: monkey at door, box at window, banana at centre\n")
for i, act in enumerate(solution, 1):
    print("Step %d: %s" % (i, act))
print("\nTotal actions in the optimal plan:", len(solution))
```

OUTPUT

```
Initial state: monkey at door, box at window, banana at centre

Step 1: Walk to the window
Step 2: Push the box to the centre
Step 3: Climb on the box
Step 4: Grasp the banana

Total actions in the optimal plan: 4
```

RESULT

The monkey reached the banana using the shortest sequence of actions found by the planner. Hence, the Monkey and Banana problem was solved successfully.

Experiment 23: Forward Chaining

Scenario: *A technical support system diagnoses computer faults by applying known rules to user inputs.*

AIM

To implement Forward Chaining and derive conclusions from the available facts.

ALGORITHM

1. Store the rules as IF (conditions) THEN conclusion and the user's observations as the initial facts.
2. Scan the rules; if all conditions of a rule are present in the fact base and its conclusion is new, fire the rule and add the conclusion to the fact base.
3. Repeat the scan until no new fact can be added (a fixed point is reached).
4. Display the facts derived in every round and the final conclusions.

PROGRAM (Python)

```
rules = [
    (["no_display", "fan_not_spinning"], "power_supply_fault"),
    (["power_supply_fault"], "replace_power_supply"),
    (["no_display", "beep_sound"], "ram_fault"),
    (["ram_fault"], "reseat_or_replace_ram"),
    (["slow_performance", "high_disk_usage"], "disk_fault"),
    (["disk_fault"], "clean_disk_or_replace_hdd"),
    (["wifi_connected", "no_internet"], "router_or_dns_fault"),
]

def forward_chain(facts):
    facts = set(facts)
    print("Initial facts:", sorted(facts))
    round_no = 0
    changed = True
    while changed:
        changed = False
        round_no += 1
        for conditions, conclusion in rules:
            if all(c in facts for c in conditions) and conclusion not in facts:
                facts.add(conclusion)
                changed = True
                print("Round %d: IF %s THEN %s" % (round_no, " AND ".join(conditions), conclusion))
    print("Final facts  :", sorted(facts))
    print()

forward_chain(["no_display", "fan_not_spinning"])
forward_chain(["no_display", "beep_sound", "slow_performance", "high_disk_usage"])
```

OUTPUT

```
Initial facts: ['fan_not_spinning', 'no_display']
Round 1: IF no_display AND fan_not_spinning THEN power_supply_fault
Round 1: IF power_supply_fault THEN replace_power_supply
Final facts  : ['fan_not_spinning', 'no_display', 'power_supply_fault', 'replace_power_supply']

Initial facts: ['beep_sound', 'high_disk_usage', 'no_display', 'slow_performance']
Round 1: IF no_display AND beep_sound THEN ram_fault
Round 1: IF ram_fault THEN reseat_or_replace_ram
Round 1: IF slow_performance AND high_disk_usage THEN disk_fault
Round 1: IF disk_fault THEN clean_disk_or_replace_hdd
Final facts  : ['beep_sound', 'clean_disk_or_replace_hdd', 'disk_fault', 'high_disk_usage', 'no_display',
    'ram_fault', 'reseat_or_replace_ram', 'slow_performance']
```

RESULT

Starting from the observed facts, new facts were derived repeatedly until no more rules could fire, giving the diagnosis of the fault. Hence, Forward Chaining was implemented successfully.

Experiment 24: Backward Chaining

Scenario: *A medical diagnosis system determines the possible disease by working backward from symptoms.*

AIM

To implement Backward Chaining and reach conclusions through goal-driven reasoning.

ALGORITHM

1. Store the rules as IF (conditions) THEN conclusion and the patient's symptoms as known facts.
2. Take the goal (hypothesis, for example "flu") to be proved.
3. If the goal is a known fact, it is proved. Otherwise, find the rules whose conclusion is the goal.
4. For such a rule, take its conditions as new sub-goals and prove each of them recursively.
5. If all conditions of a rule are proved, the goal is proved; if no rule succeeds, the goal is not proved.

PROGRAM (Python)

```
rules = {
    "viral_infection": [["fever", "cough"]],
    "flu": [["viral_infection", "body_ache"]],
    "malaria": [["fever", "chills", "sweating"]],
    "common_cold": [["sneezing", "runny_nose"]],
}
facts = {"fever", "cough", "body_ache", "sweating"}

def prove(goal, depth=0):
    indent = "   " * depth
    print(indent + "Goal:", goal)
    if goal in facts:
        print(indent + " -> known fact: TRUE")
        return True
    for conditions in rules.get(goal, []):
        if all(prove(sub, depth + 1) for sub in conditions):
            print(indent + " -> proved using rule IF", " AND ".join(conditions))
            return True
    print(indent + " -> cannot be proved: FALSE")
    return False

print("Patient symptoms:", sorted(facts), "\n")
for hypothesis in ["flu", "malaria"]:
    result = prove(hypothesis)
    print(">>> Patient has %s? %s\n" % (hypothesis, result))
```

OUTPUT

```
Patient symptoms: ['body_ache', 'cough', 'fever', 'sweating']

Goal: flu
   Goal: viral_infection
      Goal: fever
       -> known fact: TRUE
      Goal: cough
       -> known fact: TRUE
    -> proved using rule IF fever AND cough
   Goal: body_ache
    -> known fact: TRUE
 -> proved using rule IF viral_infection AND body_ache
>>> Patient has flu? True

Goal: malaria
   Goal: fever
    -> known fact: TRUE
   Goal: chills
    -> cannot be proved: FALSE
 -> cannot be proved: FALSE
>>> Patient has malaria? False
```

RESULT

Working backward from the goals, the system proved that the patient has flu and could not prove malaria because the symptom chills is missing. Hence, Backward Chaining was implemented successfully.

Experiment 25: Best First Search in Prolog (Python implementation)

Scenario: *A search application written in Prolog locates the best route using heuristic information.*

AIM

To implement Best First Search in the Prolog style (facts edge/2 and h/2 with recursive rules) and perform heuristic search. The program below expresses the Prolog facts and rules in Python.

ALGORITHM

1. Declare the facts edge(X, Y) for the connections of the map and h(X, H) for the heuristic estimate to the goal.
2. Keep an OPEN list of paths, each ending at a node; initially it contains only the start node.
3. Rule: if the best path (smallest h of the last node) ends at the goal, the search succeeds with that path.
4. Otherwise, remove the best path, extend it with every neighbour not already in the path, add the new paths to OPEN, sort OPEN by h and call the rule again (recursion).
5. If OPEN becomes empty, the search fails.

PROGRAM (Python)

```
# Facts
edge = [("s", "a"), ("s", "b"), ("a", "c"), ("a", "d"), ("b", "e"),
        ("b", "f"), ("d", "g"), ("e", "g"), ("f", "h")]
h = {"s": 10, "a": 7, "b": 6, "c": 9, "d": 3, "e": 4, "f": 8, "g": 0, "h": 5}

def neighbours(x):
    return [y for (u, y) in edge if u == x]

# Rule: best_first(Open, Goal, Path)
def best_first(open_paths, goal):
    if not open_paths:
        return None                                  # search fails
    open_paths.sort(key=lambda path: h[path[-1]])    # best path (lowest h) first
    best = open_paths.pop(0)
    node = best[-1]
    print("Best path chosen: %-22s h(%s) = %d" % (" -> ".join(best), node, h[node]))
    if node == goal:
        return best                                  # success
    for n in neighbours(node):
        if n not in best:
            open_paths.append(best + [n])
    return best_first(open_paths, goal)              # recursive call

path = best_first([["s"]], "g")
print("\nSolution path:", " -> ".join(path))
```

OUTPUT

```
Best path chosen: s                     h(s) = 10
Best path chosen: s -> b                h(b) = 6
Best path chosen: s -> b -> e           h(e) = 4
Best path chosen: s -> b -> e -> g      h(g) = 0

Solution path: s -> b -> e -> g
```

RESULT

The program repeatedly expanded the path whose last node had the lowest heuristic value and reached the goal node g. Hence, Best First Search in the Prolog style was implemented successfully.

Experiment 26: Sum of 1 to N using Recursion

Scenario: *A mathematical tutoring system computes the sum of natural numbers recursively.*

AIM

To implement the sum of the first N natural numbers (1 + 2 + ... + N) using recursion and demonstrate recursive problem solving.

ALGORITHM

1. Base case: if N = 0, the sum is 0.
2. Recursive case: sum(N) = N + sum(N - 1).
3. The function keeps calling itself with a smaller value until the base case is reached.
4. Then the values are added while the calls return; print the result and verify it with the formula N(N+1)/2.

PROGRAM (Python)

```
def sum_n(n):
    if n == 0:                # base case
        return 0
    return n + sum_n(n - 1)    # recursive case

def trace(n, depth=0):
    print("  " * depth + "sum(%d)" % n)
    if n == 0:
        return 0
    value = n + trace(n - 1, depth + 1)
    print("  " * depth + "sum(%d) returns %d" % (n, value))
    return value

print("Recursive trace for N = 4:")
trace(4)
print()
for n in [5, 10, 100]:
    print("Sum of 1 to %-3d = %-5d (formula n(n+1)/2 = %d)" % (n, sum_n(n), n * (n + 1) // 2))
```

OUTPUT

```
Recursive trace for N = 4:
sum(4)
  sum(3)
    sum(2)
      sum(1)
        sum(0)
      sum(1) returns 1
    sum(2) returns 3
  sum(3) returns 6
sum(4) returns 10

Sum of 1 to 5   = 15    (formula n(n+1)/2 = 15)
Sum of 1 to 10  = 55    (formula n(n+1)/2 = 55)
Sum of 1 to 100 = 5050  (formula n(n+1)/2 = 5050)
```

RESULT

The sum of the first N natural numbers was calculated recursively and matched the formula N(N+1)/2. Hence, recursive problem solving was demonstrated successfully.

Experiment 27: Vowel Identification using Rule-based Logic

Scenario: *A text-processing application identifies vowels in a given sentence for language analysis.*

AIM

To implement vowel identification using rule-based logic and classify characters based on predefined rules.

ALGORITHM

1. Define the rules: IF the character (in lower case) is one of a, e, i, o, u THEN it is a vowel; IF it is another letter THEN it is a consonant; IF it is a digit THEN it is a digit; otherwise it is a space/special character.
2. Read the sentence and take each character one by one.
3. Apply the rules to the character and classify it.
4. Count each class, and list the vowels with their positions.
5. Display the result.

PROGRAM (Python)

```
sentence = "Artificial Intelligence and Expert Systems"

def classify(ch):
    c = ch.lower()
    if c in "aeiou":                # rule 1
        return "vowel"
    elif c.isalpha():               # rule 2
        return "consonant"
    elif c.isdigit():               # rule 3
        return "digit"
    return "space/special"          # rule 4

counts = {"vowel": 0, "consonant": 0, "digit": 0, "space/special": 0}
vowel_positions = []
for index, ch in enumerate(sentence):
    kind = classify(ch)
    counts[kind] += 1
    if kind == "vowel":
        vowel_positions.append((ch, index))

print("Sentence :", sentence)
print("Vowels found    :", " ".join(ch for ch, _ in vowel_positions))
print("Positions       :", vowel_positions)
print("Counts          :", counts)
for v in "aeiou":
    print("  '%s' occurs %d time(s)" % (v, sentence.lower().count(v)))
```

OUTPUT

```
Sentence : Artificial Intelligence and Expert Systems
Vowels found    : A i i i a I e i e e a E e e
Positions       : [('A', 0), ('i', 3), ('i', 5), ('i', 7), ('a', 8), ('I', 11), ('e', 14), ('i', 17), ('e', 19),
    ('e', 22), ('a', 24), ('E', 28), ('e', 31), ('e', 39)]
Counts          : {'vowel': 14, 'consonant': 24, 'digit': 0, 'space/special': 4}
  'a' occurs 3 time(s)
  'e' occurs 6 time(s)
  'i' occurs 5 time(s)
  'o' occurs 0 time(s)
  'u' occurs 0 time(s)
```

RESULT

Every character of the sentence was classified using the rules and all the vowels were identified with their positions. Hence, vowel identification using rule-based logic was implemented successfully.